

Sentiment Analyzer for Product Reviews

IE 7374: Model Deployment Assignment

Group Number: 7

Team Members:

- Kiran Tamilselvan
- Manikandan Mohan
- Aniruthan Swaminathan Arulmurugan
- Manivannan Senthil Kumar
- Janani Karthikeyan
- Aravind Anbazhagan

Github Repo Link:

Working Github: <https://github.com/Mani31899/MLOPS-Sentiment-Analyzer-Project>

Old Github: <https://github.com/Aniruthan-0709/Sentiment-Analysis-Tool-for-Product-Reviews>

Video Recording Link:

Shorter Version: [Video Recording](#) [8 min]

Detailed Version: [Video Recording](#) [18 min]

Synopsis

This project presents a complete MLOps pipeline for deploying a FastAPI-based sentiment analysis model, along with a data drift detection component, on Google Kubernetes (GKE). The solution integrates CI/CD using Cloud Build and GitHub Triggers, with user interaction logging via BigQuery, and automates model retraining triggers through Pub/Sub notifications upon new model uploads GCS.

Important note:

The redeploy.ps1 and redeploydrift.ps1 files in the Model_Deployment_Pipeline folder contain commands to deploy two docker images to Google Cloud with specific to our GCP setup. When cloning and using this repository, you must modify these in it to match your own GCP project details.

Folder Structure - Model Deployment

Model_Deployment_Pipeline/

— main.py	# FastAPI web application
— model_loader.py	# GCS model loader
— gcp_logging.py	# BigQuery logging
— templates/index.html	# Web UI
— k8s.yaml	# GKE deployment and service
— cloudbuild.yaml	# CI/CD automation
data_drift_detector/	
— data_drift_check.py	# Drift detection logic
— data-drift-job.yaml	# K8s job definition for drift check

Model Deployment on Google Cloud Platform

Tools and Services Used

Google Cloud Platform (GCP)

- GKE (Google Kubernetes Engine): Orchestrates containerized microservices (FastAPI + Drift Detection) ensuring scalable, resilient deployment.
- Cloud Build: Automates CI/CD pipeline. Rebuilds and deploys new models when changes are pushed to the repository or new model artifacts are uploaded.
- Pub/Sub: Acts as the event trigger mechanism when new models are uploaded to the GCS bucket.
- BigQuery: Stores user input logs from the web app for later monitoring, dashboarding, and potential drift analysis.

Google Kubernetes Engine (GKE)

GKE is a managed Kubernetes service provided by Google Cloud Platform. It allows you to run containerized applications in a scalable and reliable manner. Usage:

- Hosts both the FastAPI sentiment app and data drift detection job.
- Ensures high availability, load balancing, and simplified orchestration for Docker containers.
- Kubernetes (kubectl): Used for deploying workloads to GKE and managing secrets, pods, and services.

Docker

Docker is a containerization platform that packages applications and their dependencies into lightweight, portable containers.

Usage:

- Dockerfiles are used to build the web-based sentiment analyzer app (Dockerfile.deploy) and the batch data drift detection job (Dockerfile.drift)
- These images are pushed to Google Container Registry and deployed on GKE.

FastAPI

FastAPI is a modern, fast (high-performance) Python web framework for building APIs with automatic OpenAPI documentation.

Usage:

- Powers the backend of the sentiment analysis web app.
- Provides HTTP endpoints to serve the model and render the HTML interface.

Frontend Website

Jinja2 is a templating engine for Python, used to generate HTML dynamically.

Usage:

- Renders the user interface of the sentiment analyzer web app.
- Displays input form, prediction results, and model version.

Google Cloud Build

Cloud Build is a CI/CD tool provided by Google Cloud for automating builds, tests, and deployments.

Usage:

- CI/CD pipeline defined in cloudbuild.yaml:
- Automatically restarts GKE deployments when new models are pushed.
- Triggers the data drift detection job.
- Waits for deployment/job completion.

Pub/Sub

Pub/Sub is a messaging service by GCP for building event-driven architectures.

Usage:

- GCS triggers Pub/Sub when a new model is uploaded.
- Cloud Build listens to this topic and starts the deployment workflow.

Google Cloud Storage (GCS)

GCS is a scalable and secure object storage service for data, model files, and reports.

Usage:

- Stores trained models (models/ directory)
- Saves generated data drift reports (drift_report/)
- Serves as a trigger source for model updates.

Google BigQuery

BigQuery is a fully managed, serverless data warehouse that supports fast SQL queries using the processing power of Google's infrastructure.

Usage:

- Stores user interaction logs: review text, sentiment predictions, and timestamps.
- Enables data analytics and is the source of live data for drift detection.

Drift Detection

Text Vectorization: A text vectorization tool from scikit-learn that converts text to a matrix of token counts.

GitHub Actions API: GitHub Actions allows you to automate workflows directly from your GitHub repo using YAML files and webhooks. Usage

- Extracts word frequencies from the reviews for both historical and live data to detect distribution changes (drift).
- `data_drift_check.py` uses GitHub REST API to trigger the data pipeline (`data_pipeline.yaml`) when drift is detected.

GCP IAM & Secrets

- Service Account JSON Key with roles: Storage Admin, BigQuery Viewer, Artifact Registry Writer, Kubernetes Engine Admin
- GKE Secrets: Securely store keys, GitHub PAT, and email credentials using Kubernetes secrets.

Deployment Pipeline Overview

1. Authentication & Setup

GCP Setup

- Enable Billing, Enable GKE, Cloud Build, Artifact Registry, BigQuery, Pub/Sub APIs
- Create:
 - GKE Cluster: sentiment-cluster
 - BigQuery Table: sentiment_data.user_logs
 - GCS Bucket: mlops_dataset123 (stores models and drift reports)
 - Pub/Sub topic: model-updates-topic to trigger rebuilds on new model uploads

```
gcloud auth login
gcloud auth configure-docker
gcloud config set project <YOUR_PROJECT_ID>
gcloud config set compute/zone us-east1-b
```

2. Infrastructure Setup

a. GKE Cluster

```
gcloud container clusters create sentiment-cluster --num-nodes=1 --zone=us-east1-b
gcloud container clusters get-credentials sentiment-cluster
```

The screenshot shows the Google Cloud console interface for a Kubernetes engine cluster named 'sentiment-cluster'. The left sidebar contains navigation options like 'All fleets', 'Resource management', 'Overview', 'Clusters', 'Workloads', 'Teams', 'Applications', 'AI/ML', 'Secrets and ConfigMaps', 'Storage', 'Object Browser', 'Upgrades', 'Backup for GKE', 'Marketplace', and 'Release notes'. The main panel displays the cluster's 'Details' tab, which includes a warning about automatic upgrades and maintenance tasks. Below this, the 'Cluster basics' section provides a table of key attributes.

Cluster basics		
Name	sentiment-cluster	
Tier	Standard	
Mode	Standard	
Location type	Zonal	
Control plane zone	us-east1-b	
Default node zones	us-east1-b	
Total size	1	
Release channel	Regular channel	
Version	1.31.6-gke.1020000	Upgrade available
Current COS version	cos-117-18613-164-28	
End of standard support	21 Dec 2025	

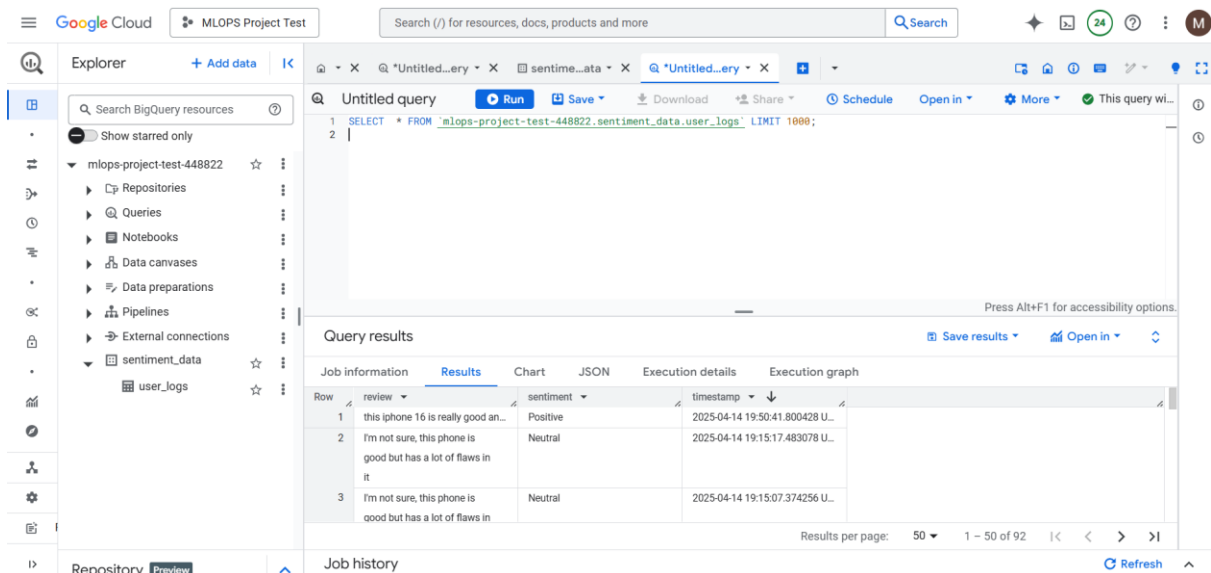
The below image is the VM instance that is automatically created when a Google Kubernetes Cluster Pods are defined. Load balancing can be set appropriately, and auto scaling is enabled so that our docker image deployed in GKE can handle high workloads. Load balancing settings can be increased further to handle network traffic based on user data

The screenshot displays the 'Load balancer details' page in the Google Cloud console. The left sidebar shows 'Network services' with options like 'Load balancing', 'Cloud DNS', 'Cloud CDN', 'Cloud NAT', 'Cloud Service Mesh', 'Service Directory', 'Cloud Domains', 'Private Service Connect', 'SSL policies', and 'Service extensions'. The main panel shows details for a 'Target-pool network load balancer' with ID 'a4d93748f4048419b9c32acc9686ae26'. It includes sections for 'Frontend' (Protocol: TCP, IP version: IPv4, IP-Port: 34.23.79.184:80, Network Tier: Premium) and 'Backend' (Name: a4d93748f4048419b9c32acc9686ae26, Region: us-east1, Health check: k8s-dac8a0d7bff03e49-node). An 'ADVANCED CONFIGURATIONS' section shows a table of instances.

Instance	Zone	34.23.79.184
gke-sentiment-cluster-default-pool-2fd9e9fb-f9lq	us-east1-b	✓

b. BigQuery Table for Logs

```
CREATE OR REPLACE TABLE `<PROJECT_ID>.sentiment_data.user_logs` (  
  review  
  STRING,  
  sentiment STRING,  
  timestamp TIMESTAMP  
);
```

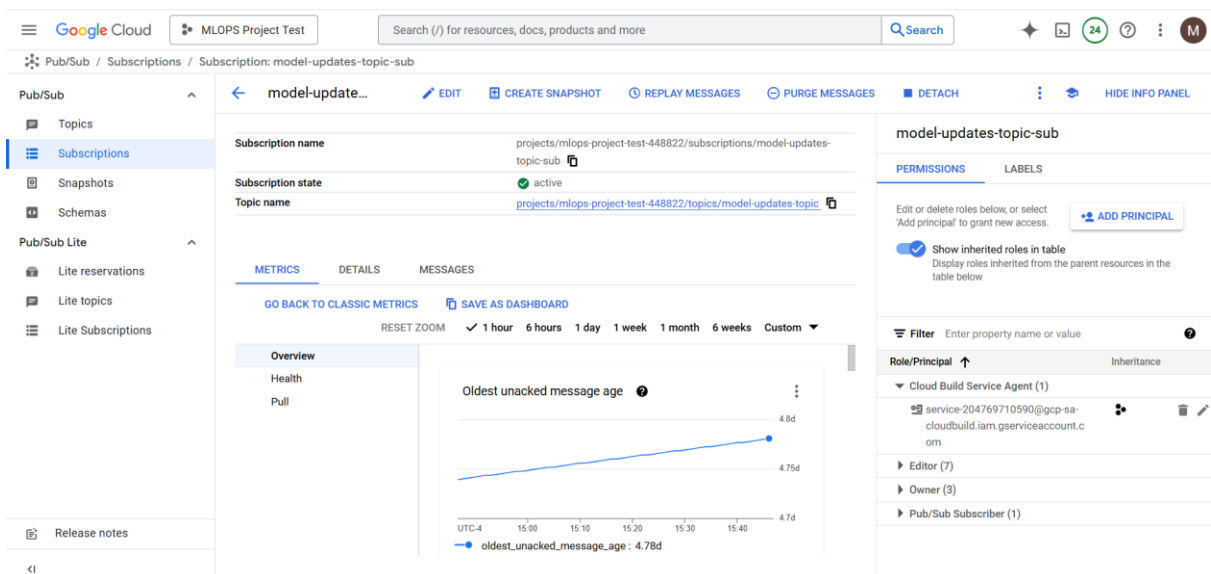


c. Pub/Sub Topic + GCS Notification

gsutil notification create -t model-updates-topic \

-f json -e OBJECT_FINALIZE -p models/ gs://<YOUR_BUCKET>

The screenshot below shows the “model-update-topic-sub” created with role/principal policy of Cloud Build and GCS model folder. This pub/sub sends a message to Cloud Build whenever there is a new model to the GCS bucket.



3. CI/CD with Cloud Build

a. Cloud Build Trigger (UI Setup)

- Trigger name: deploymodelgke
- Triggered by push or model upload
- Uses cloudbuild.yaml to:
 - Re-deploy FastAPI app
 - Trigger drift detection job

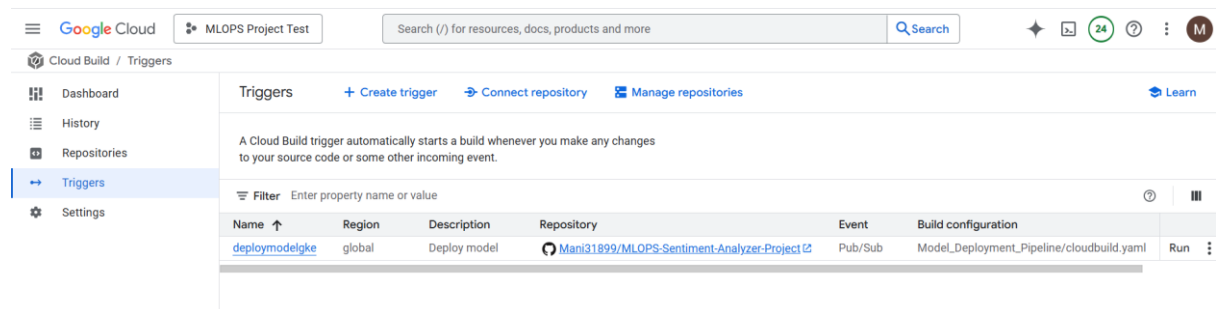
b. GitHub Repository Integration

gcloud alpha builds connections create github \

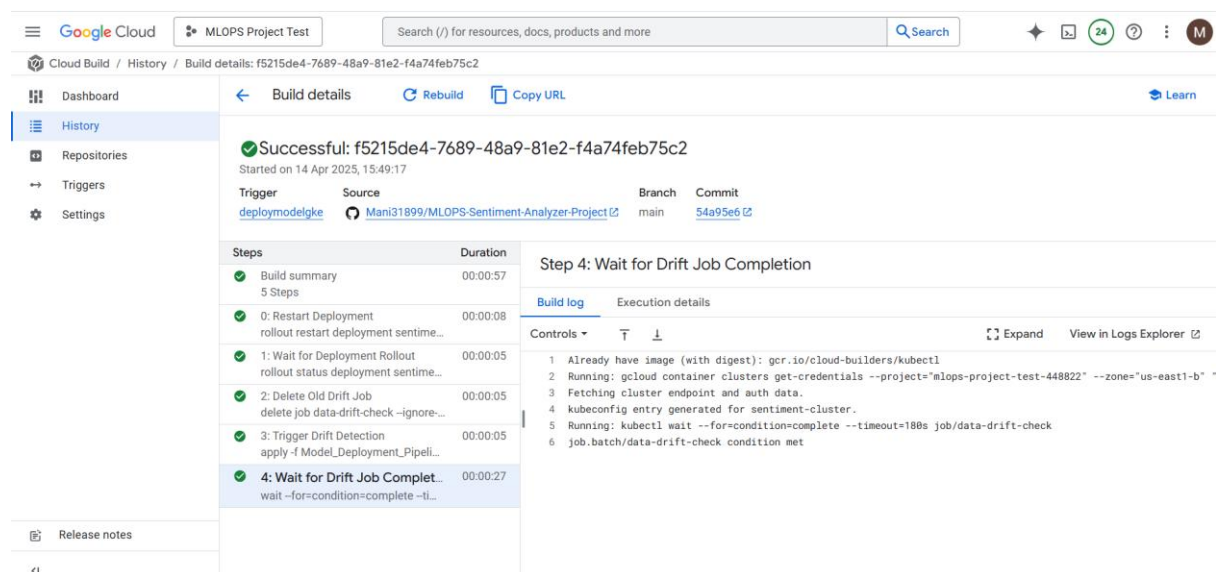
--repository-owner="<OWNER>" --repository-name="<REPO>" \

--connection-name="github-connection"

Created a Cloud Build trigger what uses the “CloudBuild.yaml” file from the github repo and orchestrates the GKE docker images.



Here, it shows the execution flow of the Cloud Build trigger using the yaml file, when Pub/Sub sends a message.



3. Build & Push Docker Images

a. Sentiment Analyzer

docker tag sentiment-analyzer-app gcr.io/<PROJECT_ID>/sentiment-analyzer-app docker

push gcr.io/<PROJECT_ID>/sentiment-analyzer-app

b. Drift Detector

```
cd data_drift_detector/
```

```
docker tag data-drift-detector gcr.io/<PROJECT_ID>/data-drift-detector docker
```

```
push gcr.io/<PROJECT_ID>/data-drift-detector
```

4. Kubernetes Secrets Setup

The required secrets are set to the GKE clusters so that it could access them. Make sure to keep the secrets in the appropriate “.secrets” folders in locally.

```
kubectl create secret generic gcp-key-secret --from-file=gcp_key.json=<path-to-key> kubectl
```

```
create secret generic github-pat --from-literal=token=<GITHUB_PAT> kubectl create secret
```

```
generic gmail-secret --from-literal=EMAIL_APP_PASSWORD='your_app_password'
```

5. Deploy to GKE

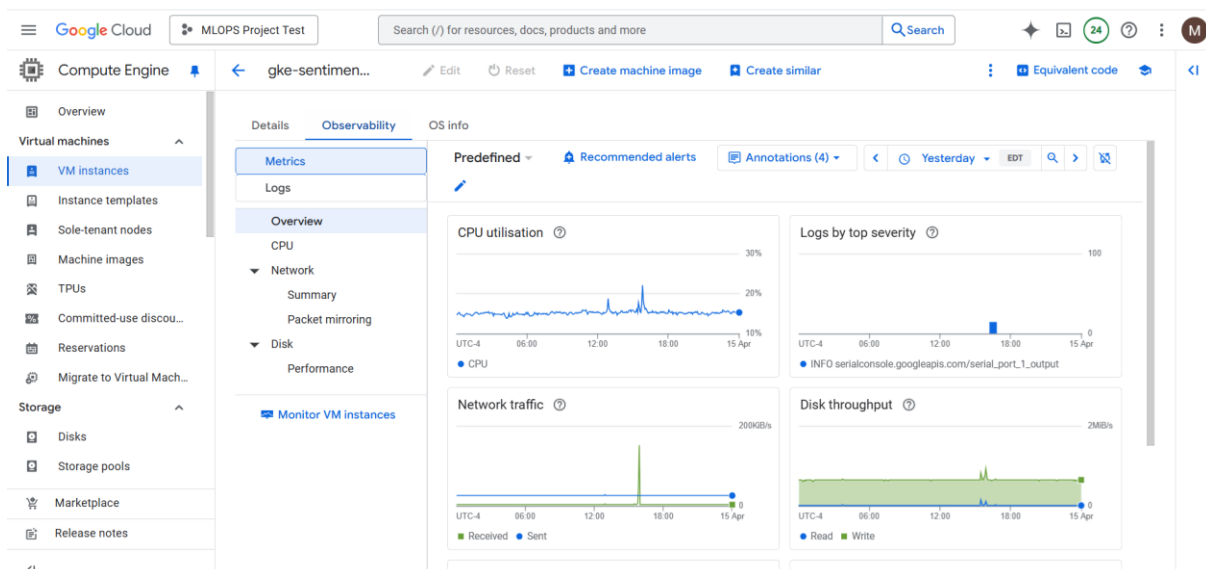
```
kubectl apply -f k8s.yaml
```

Auto-scaling and Load Testing: Auto-scaling automatically adjusts the number of VM instances in a group based on load or other metrics, ensuring that your application can handle varying amounts of traffic. Load testing simulates user traffic to ensure the system can handle real-world usage.

The startup script in the MIG will automatically activate the virtual environment and start the FastAPI server. You can SSH into a VM instance from the MIG to check the logs and verify that the server is running. Check the log file /tmp/startup.log for any errors.

Test the load balancer: Send HTTP requests to the load balancer's IP to the /predict/ endpoint with sample reviews.

Monitor Auto-scaling: With the simulated load of 200 users and a spawn rate of 10 requests per second, observe scaling to 8 instances. Monitor the instances and performance through the GCP Console to see if new instances are created and traffic is balanced.



6. Access the Application

`kubectl get service sentiment-service`

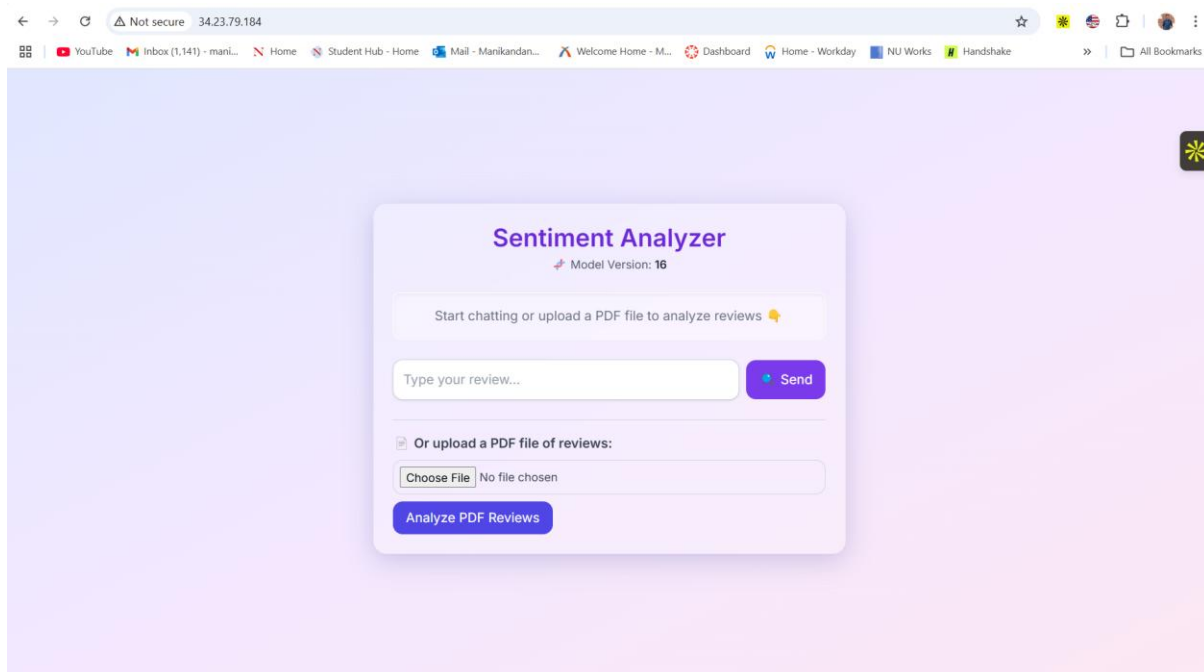
We the external-IP using this command where we can access the web application externally.

```
Re-applying Kubernetes YAML (if updated)...
deployment.apps/sentiment-analyzer unchanged
service/sentiment-service unchanged
Fetching External IP of LoadBalancer...
NAME                TYPE           CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
sentiment-service   LoadBalancer  34.118.226.19   34.23.79.184     80:30535/TCP     3d23h
```

7. Flask API - UI Page

The UI for our application has been developed using flask API served. The UI after being hosted on the GKE clusters with custom VM image and load balancer looks like this. Here the user reviews of the products is the input, and the model outputs the sentiment of the review. The model version used is 10 and it is updated and deployed each time a new model is created. Anybody can access this link and use out Sentiment Analyzer website

Website link: <http://34.23.79.184/>



8. Drift Monitoring & Logs

`kubectl get pods`

`kubectl logs <data-drift-pod-name>`

- Monitor the logs for drift detection results.
- All app logs and predictions are stored in BigQuery for future analysis.


```
secret/gcp-key-secret configured
Creating GitHub Secret...
secret/github-pat configured
Creating Gmail Secret...
secret/gmail-secret configured
Applying Kubernetes Job to run drift detection...
job.batch/data-drift-check unchanged
Job submitted. Check logs using Pod Name using:
```

9. Model Update & Drift Detection Automation

When a new model is uploaded to GCS (models/ folder), Pub/Sub triggers Cloud Build.
Cloud Build pipeline:

- Pulls the new model
- Updates deployment
- Runs data-drift-detector job
- Logs data drift report as html in GCS data

Data Drift Report

Drift Detected: False
Drifted Features: 56 / 83197

Cloud Storage

Bucket details

Go to path Refresh Learn

Overview

Buckets

Monitoring

Settings

Storage Intelligence

Insights datasets

Configuration

mlops_dataset123

Location: us (multiple regions in United States) Storage class: Standard Public access: Not public Protection: Soft delete

Objects Configuration Permissions Protection Lifecycle Observability New Inventory Reports Operations

Folder browser

Buckets > mlops_dataset123 > drift_report

Create folder Upload Transfer data Other services

Filter by name prefix only Filter objects and folders Show Live objects only

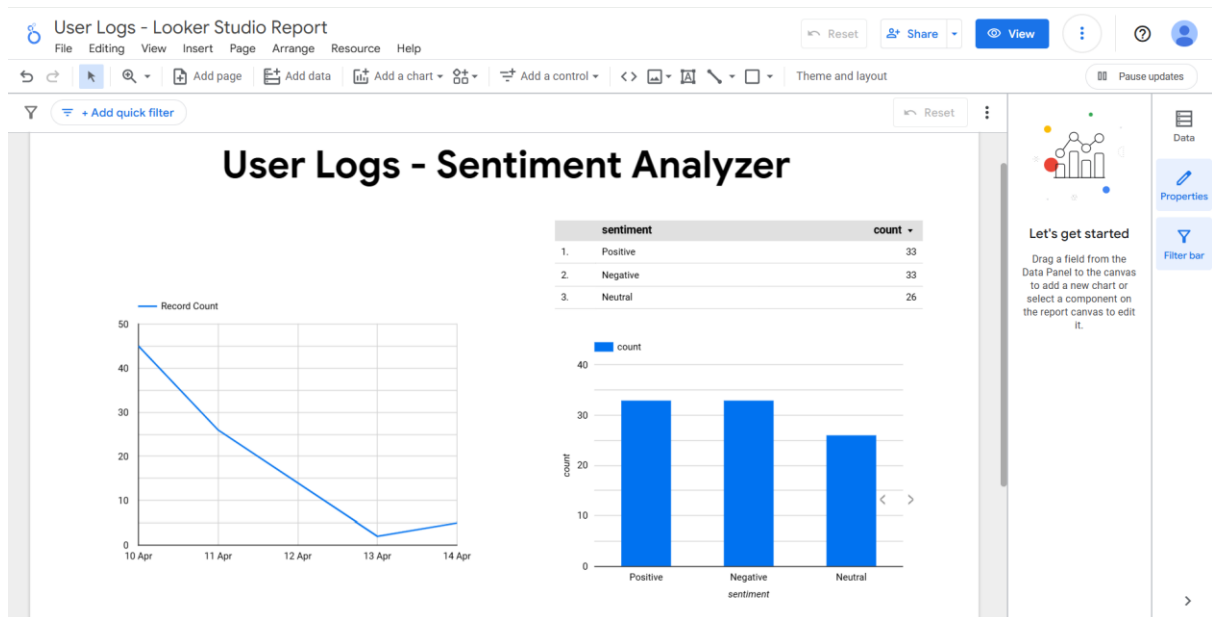
Name	Size	Type	Created	Storage class
drift_report.html	462 B	text/html	14 Apr 2025, 15:50:09	Standard

- 107
- 21
- 38
- amd
- an
- and
- average
- bad
- bag
- best
- bough
- bought
- br
- day
- deteriorated
- flaws
- food
- for
- good
- had

10. Dashboarding

Integrating Looker Studio to BigQuery with Monitoring Dashboards:

- Monitor user interactions
- Track model sentiment predictions
- Detect data drift visually



Overview

This MLOps pipeline enables fully automated, scalable model deployment using GCP's native services. Key benefits include:

- Auto-deploy on model change (via Cloud Build + Pub/Sub)
- Containerized architecture (Docker + GKE)
- Secure secret management (Kubernetes secrets)
- Logging + monitoring (BigQuery, Cloud Logs)
- Drift Detection to proactively retrain models if needed

Automated CI/CD Integration

This project utilizes GitHub Actions to implement Continuous Integration and Continuous Deployment (CI/CD) for the model pipeline. This automation ensures that any changes made to the main branch trigger the model pipeline, facilitating seamless updates and deployments.

- Data Pipeline yml file : `.github/workflows/ data_pipeline.yml`
- Model Development yml file : `github/workflows/ data_pipeline.yml`
`train_trackmodel_upload.yml`
- Pub_Sub + CloudBuid + GKE: Detects model push in GCS and deployed to GKE

The CI/CD workflow is designed to:

- Automatically triggers the data pipeline it in turn further triggers the model training and evaluation process whenever changes are pushed to the main branch.
- GCP services ensures that the latest version of the model is always tested and deployed, maintaining high code quality and performance.

Step 1: Set Up a Service Account

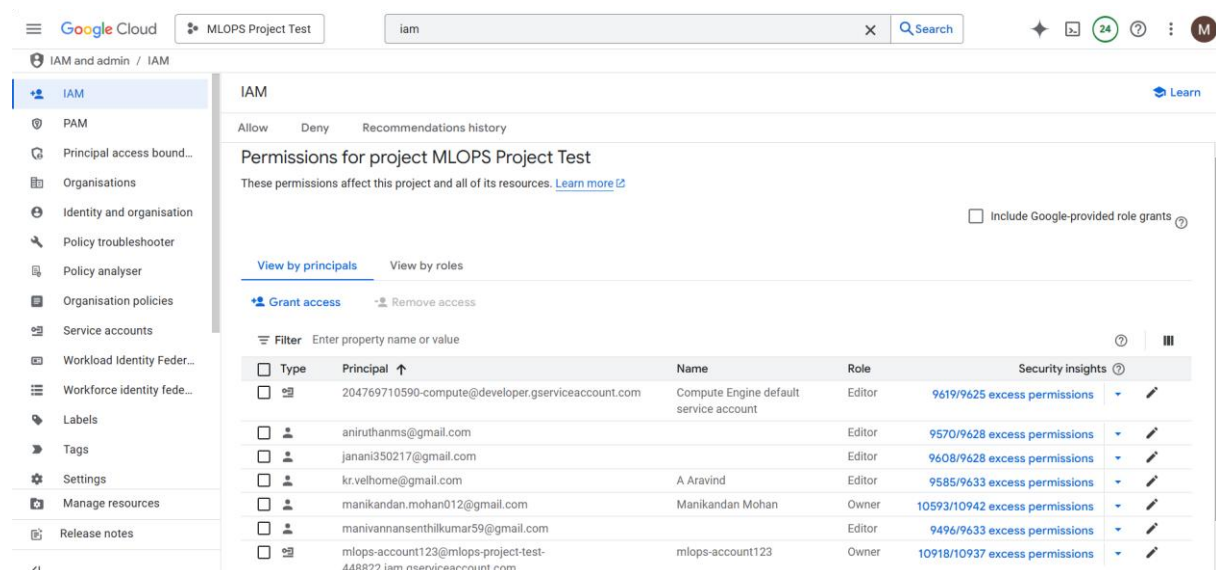
Go to the GCP Console:

- Navigate to the IAM & Admin → Service Accounts.
Create a New Service Account:
- Click + Create Service Account.
Fill in the Service Account Name and Description.
- Click Create and Continue.

Assign Roles: Go to the Service Account Details: Find your newly created service account under Service

Accounts. Click the Email of the service account. Add IAM Policies: Click Permissions → Grant Access → Add Another Role. Assign roles based on the CI/CD pipeline needs.

Service Account key: Generate any key specific to the service account. This gcp_key.json needs to be stored at appropriately in Github secrets, GKE secrets so that it could use the GCP services.

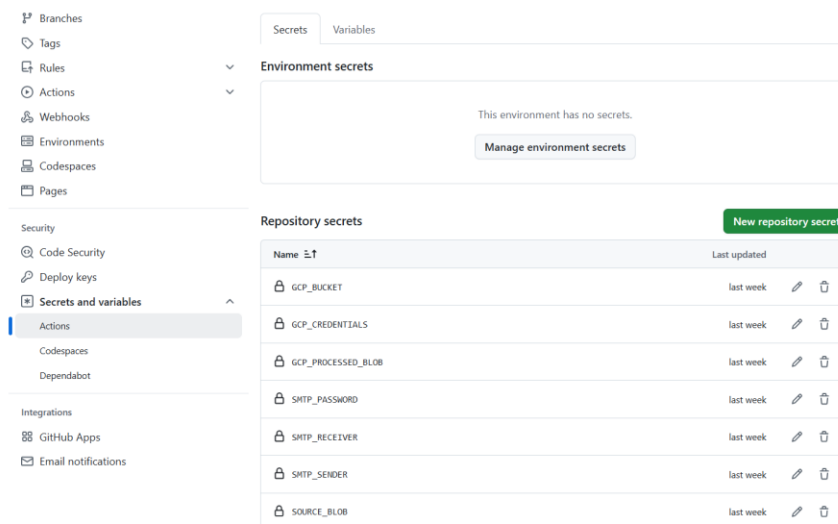


The screenshot shows the Google Cloud IAM console for project 'MLOPS Project Test'. The left sidebar lists various IAM and admin tools. The main content area is titled 'Permissions for project MLOPS Project Test' and shows a table of granted access. The table includes columns for Type, Principal, Name, Role, and Security insights. The table lists several principals, including a Compute Engine default service account and several individual users, all with Editor or Owner roles. A filter bar is visible above the table, and a 'Grant access' button is present.

Type	Principal	Name	Role	Security insights
	204769710590-compute@developer.gserviceaccount.com	Compute Engine default service account	Editor	9619/9625 excess permissions
	aniruthanms@gmail.com		Editor	9570/9628 excess permissions
	janani350217@gmail.com		Editor	9608/9628 excess permissions
	kr.velhome@gmail.com	A Aravind	Editor	9585/9633 excess permissions
	manikandan.mohan012@gmail.com	Manikandan Mohan	Owner	10593/10942 excess permissions
	manivannansenthikumar59@gmail.com		Editor	9496/9633 excess permissions
	mlops-account123@mlops-project-test-448822.iam.gserviceaccount.com	mlops-account123	Owner	10918/10937 excess permissions

Step 2: Add the Key to GitHub Secrets

Go to Your GitHub Repository: Navigate to your repository → Settings → Secrets and variables → Actions → New repository secret. Add a New Secret: Name the secret (e.g., GCP_SERVICE_ACCOUNT_KEY). Copy and paste the contents of the JSON key file into the secret. Use the Secret in your Github Actions Workflow. Add a step in your Github Actions workflow to authenticate with GCP using the service account key and SMTP keys for email alert.



Step 3: Create the necessary GCP services

Create GCP services such as Pub/Sub, Cloud Build, Google Kubernetes and Big Query table using the gcloud SDK so that the new model gets deployed to the website whenever the “Model Development Pipeline” creates a new model in GCS.

```

1 #Download google cloud sdk from "https://cloud.google.com/sdk/docs/install"
2 #set it to PATH in environment variables
3
4 # Make sure to have gcp_key.json in two places
5 # In Model_Deployment_Pipeline/gcp_key.json
6 # In Model_Deployment_Pipeline/data_drift_detector/gcp_key.json
7
8 #Authenticate Google Cloud Account and set Project
9 gcloud auth login
10 gcloud config set project <YOUR_BUCKET_ID>
11
12 #Create a GKE cluster and authenticate it
13 gcloud container clusters create sentiment-cluster --num-nodes=1 --zone=us-east1-b
14 gcloud container clusters get-credentials testsentiment-cluster --zone=us-east1-b
15
16 #Create a Big Query Table to store the user logs"
17 bq query --use_legacy_sql=false '
18 CREATE OR REPLACE TABLE <YOUR_BUCKET_ID>.sentiment_data.user_logs (
19   review STRING,
20   sentiment STRING,
21   timestamp TIMESTAMP
22 )'
23
24 #Authenticate your Github Repo with GCP Cloud Build
25 gcloud alpha builds connections create github --repository-owner="YOUR_GITHUB_USERNAME" --repository-name="YOUR_REPO_NAME" --conn
26
27 #Create a Cloud Build Trigger that triggers the GKE cluster
28 gcloud beta builds triggers create github --name="deploymodelgke" --region=us-east1 \
29   --repo-name="YOUR_REPO_NAME" \
30   --repo-owner="YOUR_GITHUB_USERNAME" \
31   --branch-pattern="*/main*" \
32   --build-config="cloudbuild.yaml" \
33   --include-logs-with-status \
34   --substitutions="_DEPLOY_ENV=prod"
35
36
37 #Create a Pub/Sub topic to trigger Cloud Build if there is Model Push in GCS
38 gsutil notification create -t model-updates-topic -f json -e OBJECT_FINALIZE -p models/ gs://<YOUR_BUCKET_ID>
39
40

```

Step 4: CI/CD Implementation

The data pipeline is trigger with a code change in main branch, the model pipeline is automatically triggered. When the model pipeline pushes a new model in the GCS, Pub/Sub sends a message to Cloud Build trigger which in turn deploys the model to GKE clusters.

Data Pipeline Workflow:

Mani31899 / MLOPS-Sentiment-Analyzer-Project

Code Issues Pull requests **Actions** Projects Wiki Security Insights Settings

← Data Pipeline

✓ **Data Pipeline #158** Re-run all jobs ...

Summary

Jobs

run-mlops-pipeline

Run details Usage Workflow file

run-mlops-pipeline
succeeded yesterday in 2m 11s

Search logs

- Set up job 2s
- Checkout repository 1s
- Set up Python 0s
- Install dependencies 1m 35s
- Write GCP Credentials from Secret 0s
- Data Ingestion & DVC Hashing 1s
- Data Preprocessing 6s
- Schema Validation 5s
- Anomaly Detection 5s
- Bias Detection 5s
- Run Unit Tests 3s

Model Development Pipeline Workflow:

Mani31899 / MLOPS-Sentiment-Analyzer-Project

Code Issues Pull requests **Actions** Projects Wiki Security Insights Settings

← Model Development Pipeline

✓ **trigger-model-pipeline #172** Re-run all jobs ...

Summary

Jobs

train-and-upload

Run details Usage Workflow file

train-and-upload
succeeded yesterday in 1m 44s

Search logs

- Set up job 1s
- Checkout Repository 1s
- Set Up Python 0s
- Install Dependencies 39s
- Authenticate with Google Cloud 12s
- Create Directories 0s
- Download Data from GCP 2s
- Train the Model 11s
- Hyperparameter Tuning 10s
- Validate the Model 3s
- Detect Model Bias 3s

Model Deployment by Cloud Build Trigger:

The screenshot shows the Google Cloud Build interface for a project named 'MLOPS Project Test'. The build details page shows a successful build with ID 'f5215de4-7689-48a9-81e2-f4a74feb75c2' started on 14 Apr 2025, 15:49:17. The build was triggered by a 'deploymodelgke' trigger from the source 'Mani31899/MLOPS-Sentiment-Analyzer-Project' on the 'main' branch with commit '54a95e6'. The build steps are listed in a table:

Steps	Duration
Build summary 5 Steps	00:00:57
0: Restart Deployment rollout restart deployment sentime...	00:00:08
1: Wait for Deployment Rollout rollout status deployment sentime...	00:00:05
2: Delete Old Drift Job delete job data-drift-check -ignore...	00:00:05
3: Trigger Drift Detection apply -f Model_Deployment_Pipeli...	00:00:05
4: Wait for Drift Job Complet... wait -for=condition=complete -ti...	00:00:27

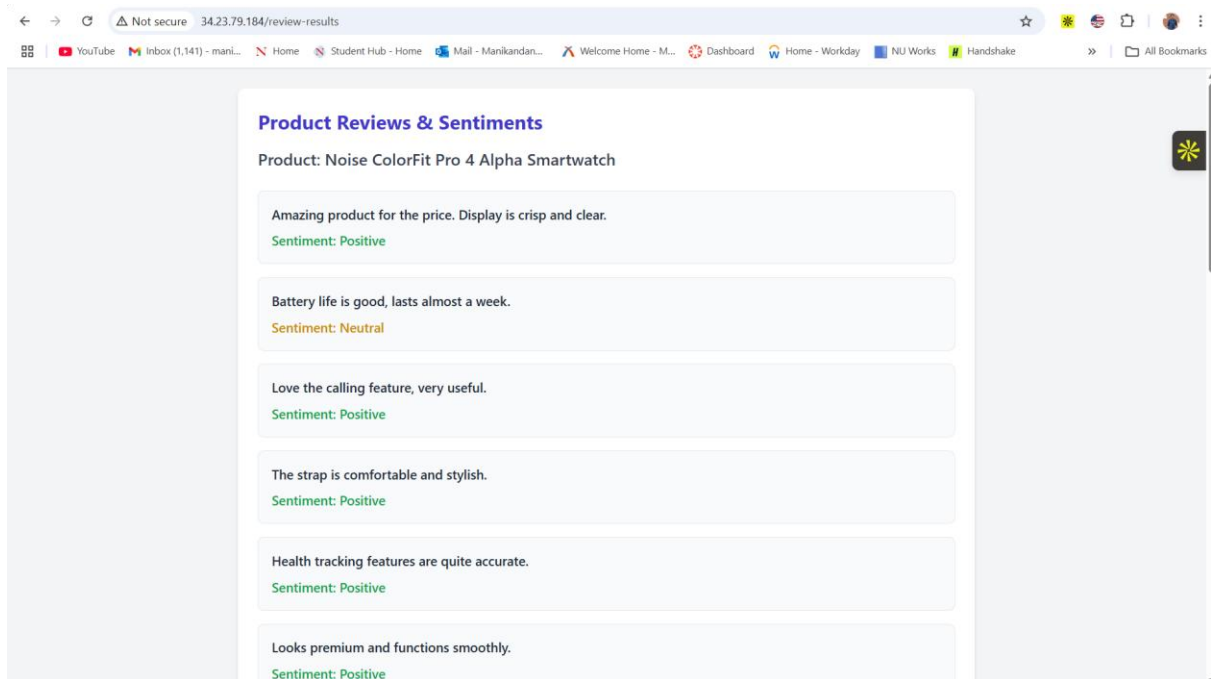
The 'Step 4: Wait for Drift Job Completion' is expanded, showing the build log:

```
1 Already have image (with digest): gcr.io/cloud-builders/kubectl
2 Running: gcloud container clusters get-credentials --project="mlops-project-test-448822" --zone="us-east1-b"
3 Fetching cluster endpoint and auth data.
4 kubeconfig entry generated for sentiment-cluster.
5 Running: kubectl wait --for=condition=complete --timeout=180s job/data-drift-check
6 job.batch/data-drift-check condition met
```

Below is an example where the model is deployed to an External-IP (<http://34.23.79.184/>) and correctly predicted the “Neutral” sentiment of the review entered. Anybody can access this link and use out Sentiment Analyzer website
Website link: <http://34.23.79.184/>

The screenshot shows a web browser displaying the 'Sentiment Analyzer' application at the IP address 34.23.79.184. The application has a purple header with the title 'Sentiment Analyzer' and 'Model Version: 16'. The main content area has a light purple background. A text input box contains the review: 'You: This iphone model is good but has a few flaws in it.' Below the input, the bot's response is shown: 'Bot: Neutral' with a 'Confidence: 90%'. A 'Send' button is next to the input box. Below this, there is a section for uploading a PDF file of reviews, with a 'Choose File' button and a file named 'Smartwatch_Product_Reviews.pdf' selected. An 'Analyze PDF Reviews' button is at the bottom.

There are two options, either one could enter the review manually in the box to get the sentiment of the text or one can also upload a “Product Reviews” pdf (in a particular format) to get the reviews analyzed in a detailed way. The pdf format should be consist of Product name, details features mentioned and importantly under “User Reviews:” section, it should contain all the reviews for that particular product. Our model extracts those reviews and performs sentiment analysis on all of the reviews and shows the output as below.

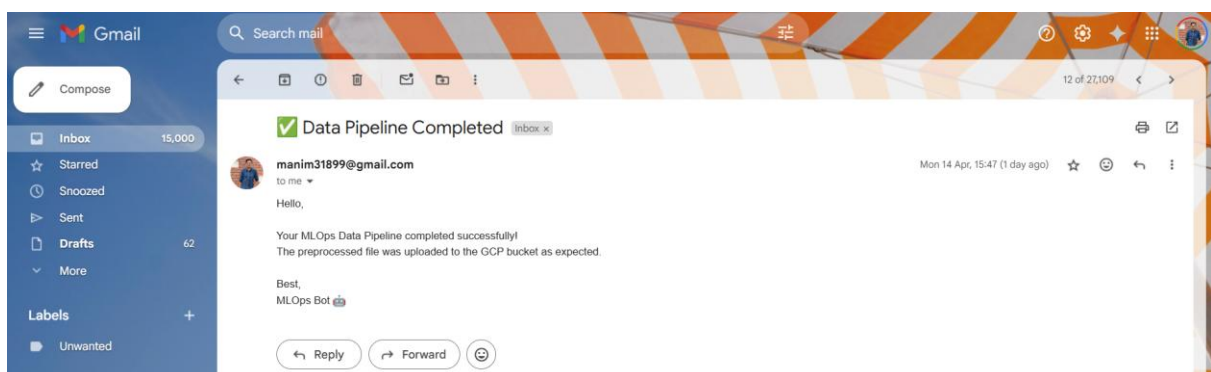


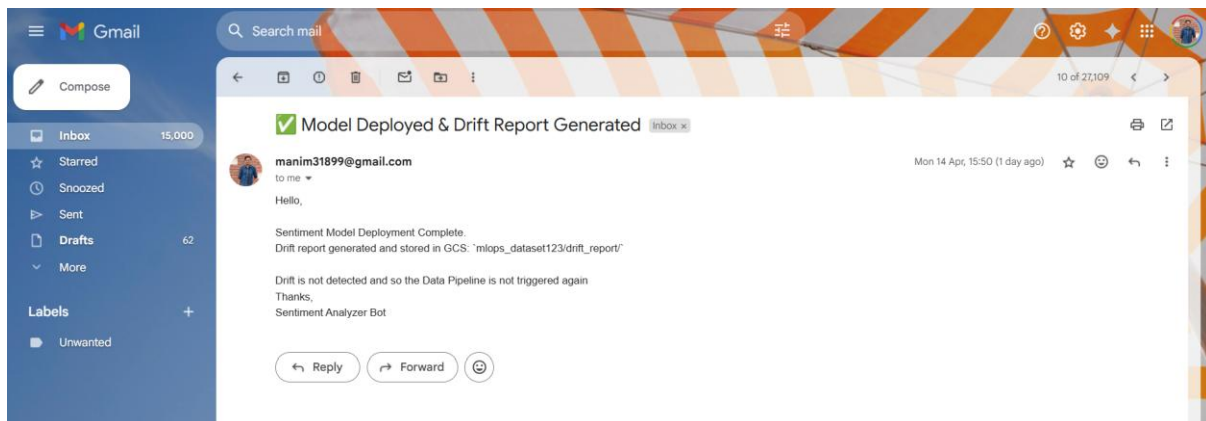
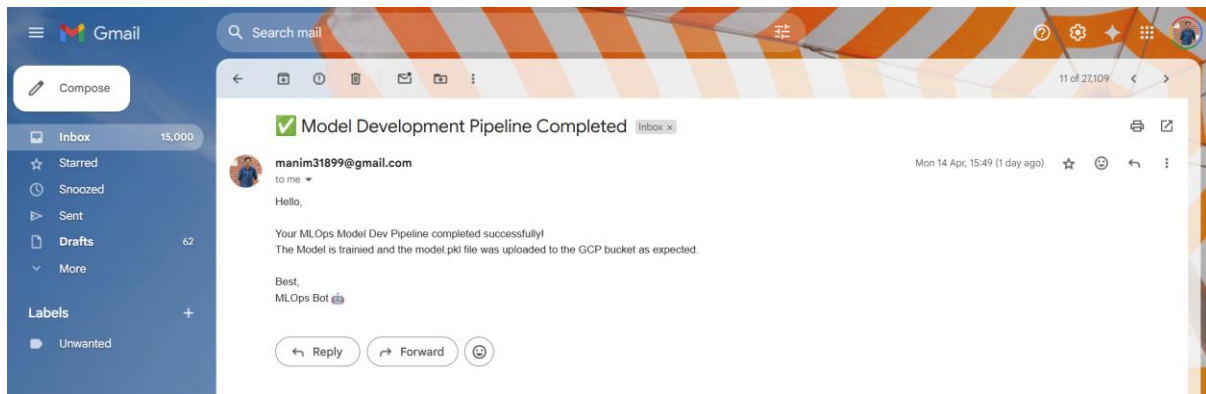
Step 5: Logging and Monitoring

We have implemented logging and monitoring using BigQuery and Google Cloud Monitoring to capture and analyze data from website such as user logs and model prediction. When the user enters input features, the model predicts the sentiment of the text as positive, negative and neutral. The input values, predicted output, and related metadata are logged to BigQuery and Cloud Monitoring for auditing, performance tracking, and error analysis. Looker studio is used to visually monitor the website and user data through big query.

Step 6: Setup Email Alerts

When each pipeline completes, set up a email alert to an email of your choice using SMTP protocol.





Conclusion

The MLOps deployment pipeline establishes a robust, scalable infrastructure for the Sentiment Analyzer prediction model on GCP. It deploys the model into a website that offers key benefits such as auto-scaling to handle varying loads, comprehensive logging and monitoring for transparency and debugging, cost-effective resource utilization, and reliable performance tracking through custom dashboards and alerting policies. This approach provides a solid foundation for deploying and managing machine learning models at scale, making it an effective solution for E-commerce companies and product owners on such websites to analyze the sentiment of the users entered reviews for their products.

Future Scope: AI-Powered Chatbot for Product Insights

To extend the functionality of the sentiment analysis platform, we plan to develop an AI-powered chatbot interface for product owners. This feature will allow users to enter a product ID and ask natural language questions like “What do customers say about this product?” and “How does it compare with others in the same category?”

- **Conversational Q&A:** Uses LLMs (e.g., OpenAI or Gemini) to summarize and answer review-based queries via a chatbot UI.
- **Product Comparison:** Benchmarks sentiment trends, review counts, and top keywords against other products in the same category.
- **Review Summarization:** Highlights strengths and weaknesses from user reviews (e.g., price, quality, delivery).
- **Trend Analysis:** Displays sentiment changes over time through simple visualizations.
- **Optional Graph Reasoning:** Uses a product knowledge graph (e.g., Neo4j) to support structured queries and advanced reasoning.